Lesson 13: A Hit That Reacts

Godot Engine

VIDEO GAME CREATION

This week: getting hit finally LOOKS like getting hit. A brief flash, then a small explosion, before the enemy disappears.

Open your own [YourName]Game project from Lesson 12 - don't start a new one.

Important: your score, lives, wave, and win/lose logic should all still work exactly as before. Today only changes how the enemy's removal LOOKS, not when your game-state actually changes.

Setup

1. Open your [YourName]Game project from Lesson 12.
2. Create a new small scene, explosion.tscn: a Sprite2D (Texture = explosion_placeholder.png) with a Timer child node (Wait Time = 0.2, One Shot on).
3. On the explosion scene's own script, connect the Timer's timeout signal to a function that calls queue_free() - the explosion cleans itself up, with no help needed from the Enemy or main scene.
4. Import enemy_invader_white.png (this lesson's Assets folder) into your project - it's the same enemy silhouette as Lesson 4's sprite, pre-recolored solid white, used below for the hit flash.

Level 1 - Minimal (MVP)

On every hit, the enemy briefly flashes, and a small independent explosion effect appears and cleans itself up. Score, lives, wave, and win/lose logic are all unaffected.



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.

5. On the Enemy scene's script, find the function that currently detects a bullet hit (from Lesson 4) - it likely calls queue_free() and emits a "destroyed" signal immediately.
6. Right after detecting the hit, instantiate explosion.tscn at global_position and add it to the scene (code below) - this happens immediately and independently, with zero effect on anything else.
7. Swap the Enemy's texture to enemy_invader_white.png for the flash, then keep the existing destroyed.emit() call running immediately after that, unchanged - score/wave/win-check must happen at exactly the same instant as last week. Only THEN await get_tree().create_timer(0.1).timeout to pause this function for a tenth of a second, with queue_free() moved to after the await. A color tint (modulate) won't work here: modulate multiplies the sprite's existing color, and multiplying any color by white leaves it completely unchanged - a genuinely white sprite is the only way to get a visible white flash on colored source art, the same reason Lesson 2's Recolor swaps sprites instead of tinting them.
8. queue_free() is the only line that actually moves - it's the one thing that should wait.
9. Run the scene and play through a full round - each hit should flash the enemy white briefly, and a small burst should appear at that spot for a moment.

```
# Enemy's hit-detection function (revised):
const FLASH_TEXTURE = preload("res://enemy_invader_white.png")

func _on_area_entered(area):
    if area.is_in_group("bullets"):
```

```
        area.queue_free()
        var explosion = explosion_scene.instantiate()
        explosion.global_position = global_position
        get_tree().current_scene.add_child(explosion)
        texture = FLASH_TEXTURE
        destroyed.emit()
        await get_tree().create_timer(0.1).timeout
        queue_free()
```

Level 2 - Complete (Alpha/Beta)

The flash and explosion play correctly on every single hit, with no leftover explosions, no enemies stuck mid-flash, and no change to score/lives/wave timing.

- Play a full round: every single hit shows a flash and an explosion, with no enemy left stuck mid-flash.
- Confirm score, wave-clear, and win/lose all still happen at exactly the moments they did last week - destroyed.emit() runs before the await, so GameManager hears about the hit immediately; only queue_free() is delayed.
- Confirm explosion.tscn instances never pile up - each one removes itself via its own Timer, with no help from the Enemy.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: instead of one flat instant swap, alternate texture between FLASH_TEXTURE and the Enemy's normal texture 2-3 times across the 0.1 seconds (with tiny awaits between) for a flicker instead of a single snap.
- Visual: scale the explosion up slightly over its 0.2-second lifetime using a Tween, so it feels like it's expanding rather than just appearing and vanishing.
- Structure: notice explosion.tscn knows nothing about score, lives, or waves - it's a fully self-contained scene, the same discipline as this course's bullets and enemies.

If Something's Wrong

- Enemy vanishes instantly, no flash visible: confirm await get_tree().create_timer(0.1).timeout was actually added, and that queue_free() comes AFTER it (destroyed.emit() should stay BEFORE the await, not move with it).
- Enemy disappears but never visibly turns white: confirm enemy_invader_white.png was actually imported and FLASH_TEXTURE's preload path matches where it landed in the project - a missing/misnamed path fails silently and the sprite just keeps its normal texture.
- Explosion never appears, or an error about a missing scene: confirm explosion_scene is preloaded/exported and assigned to the actual explosion.tscn file.
- Two hits on the same enemy cause an error: confirm area.queue_free() (the bullet) happens before the await, so a second bullet can't re-trigger this same function on an enemy already mid-flash.

Want More?

Student Resource List > Godot section, or docs.godotengine.org